Contents

IaC — Interview Questions & Answers **1**

IaC — Interview Questions & Answers

Q1: What is Terraform state and why is it important? > State is a JSON file mapping Terraform resources to real-world infrastructure. It stores resource IDs, metadata, and dependency order. Without state, Terraform can't know what exists vs what to create/update/delete. > Remote state (S3 + DynamoDB locking) is mandatory for teams — prevents concurrent applies that corrupt state.

Q2: What happens during `terraform plan`? > 1. Reads current state file > 2. Calls provider APIs to refresh real resource state (unless `-refresh=false`) > 3. Computes diff: desired (config) vs actual (state) > 4. Shows create/update/destroy actions > No changes to real infrastructure. Save plan with `-out=tfplan` for deterministic apply.

Q3: How do you manage secrets in Terraform? > Never store secrets in `.tf` files or state. > Options: > - Mark outputs as `sensitive = true` (hidden in CLI, still in state) > - Use `aws_secretsmanager_secret_version` data source at runtime > - Pass via env vars: `TF_VAR_db_password=$SECRET` > - Use Vault provider to read secrets dynamically > - Encrypt state at rest (S3 SSE + DynamoDB encryption)

Q4: Explain `terraform import` and its limitations. > Brings existing infrastructure under Terraform management by writing its state. > `terraform import aws_instance.web i-1234567890abcdef0` > Limitations: only imports state, does NOT generate `.tf` config. You must write the config manually to match. Use `terraform state show` after import to see attributes.

Q5: What is the difference between `count` and `for_each`? > `count`: integer, creates indexed list. Removing item in the middle shifts all subsequent indexes → causes destroy/recreate. Good for identical resources. > `for_each`: map or set, creates named instances. Removing one item only destroys that item. Better for heterogeneous resources with unique identifiers.

Q6: How do you handle Terraform in a CI/CD pipeline? > Pipeline stages: > 1. `terraform fmt -check` — formatting validation > 2. `terraform validate` — syntax check > 3. `tflint` + `checkov`/`tfsec` — linting and security scanning > 4. `terraform plan -out=tfplan` — on PR (post plan as PR comment via atlantis/terraform-github-actions) > 5. `terraform apply tfplan` — on merge to main, require approval > Use Atlantis or Terraform Cloud for plan/apply workflow with PR integration.

Q7: What is the Terraform `lifecycle` block? Explain `prevent_destroy` and `ignore_changes`. > `prevent_destroy = true` — block any operation that would destroy this resource. Protects databases, certificates. > `ignore_changes = [field]` — don't track drift on specific fields (e.g., `ami` that gets updated externally, `password` managed by rotation). > `create_before_destroy = true` — create replacement before destroying original. Needed for resources that can't have downtime.

Q8: Ansible is "agentless" — how does it communicate with nodes? > Uses SSH for Linux (paramiko or native ssh binary). Uses WinRM for Windows. No agent installed on managed nodes. > Just needs: SSH access, Python (2.7 or 3.x) on target. > Connection settings: `ansible_connection`, `ansible_user`, `ansible_ssh_private_key_file` > For containers/K8s: uses `kubectl` or `docker` connection plugin.

Q9: What is idempotency and how does Ansible achieve it? > Idempotency: running an operation multiple times has the same effect as running it once. > Ansible modules check current state before acting: > - `ansible.builtin.file` checks if path exists before creating > - `ansible.builtin.package` checks if package installed before installing > - `ansible.builtin.template` computes checksum — only copies if content changed > Non-idempotent: `ansible.builtin.command` and `ansible.builtin.shell` — use `creates` or `removes` args, or check with `when`.

Q10: How do you structure Terraform for multiple environments (dev/staging/prod)? > Pattern 1 — Workspaces: simple but all envs share same state backend path structure > Pattern 2 — Directory per env (recommended for large orgs):

```
environments/
├── dev/
│   ├── main.tf
│   └── terraform.tfvars
├── staging/
```

```
│   ├── main.tf
│   └── terraform.tfvars
└── prod/
    ├── main.tf
    └── terraform.tfvars
modules/
├── vpc/
├── eks/
└── rds/
```

Pattern 3 — Terragrunt with `find_in_parent_folders()` for DRY remote backend config.

Q11: Explain Terraform `depends_on` — when is it necessary? > Terraform infers dependencies automatically from resource references. `depends_on` is only needed for implicit dependencies not captured in config — e.g., an EC2 instance depends on an IAM policy being attached (but doesn't reference it in code), or a Helm chart depends on K8s namespace existing via a separate resource. > Overuse of `depends_on` slows parallelism — only use when necessary.

Q12: How do you handle Terraform state corruption or drift? > **Drift detection**: `terraform plan -refresh-only` shows drift without planning changes > **State manipulation**: > - `terraform state list` — show all resources > - `terraform state show <resource>` — inspect resource > - `terraform state mv` — rename resource (avoid destroy/recreate on refactor) > - `terraform state rm <resource>` — remove from state without destroying real resource > - `terraform force-unlock <lock-id>` — release stuck lock (use carefully) > **Corruption**: restore from S3 versioned backup of tfstate file